Learning an Agent to Play Minesweeper

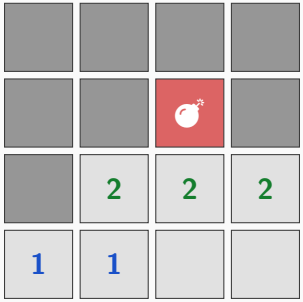
Standard RL Project

Cristiano Corsi ▪ Lorenzo Ceccanti

Symbolic and Evolutionary Artificial Intelligence (SEAI)
University of Pisa
https://github.com/lorenzoceccanti/minesweeper-rl-agent

Minesweeper board as a customized MDP

- Grid of $H \times W$ cells, N hidden mines; revealed safe cells show the count of adjacent mines (0–8). **Flags are not considered** in this environment.
- Agent's goal: uncover every safe cell without ever selecting a mine.



Hidden Revealed / number Mine that ended the episode

A 4×4 example after a loss. Three mines sit on this board.

MDP modelling: the action space

Every Python class extending the `gym.Env` base class must have the attributes `action_space` and `observation_space` used to redefine A and S.

- **Action space** - `gym.spaces.Discrete`. The action space on a board depends on the board H and W. It's an integer number, between 0 and `(H` $\times$ `W) - 1`.
- Inside the `step()` method the action is mapped to a board cell by doing: $i = a \,//\, W, \quad j = a \bmod W$.
- We could have chosen a `gym.spaces.MultiDiscrete` to directly represent an action as a tensor containing directly $[xy]$ coordinates, but this representation adapts more to DQN, who produces as much as $Q(s, a)$ as many are the possible actions.

MDP modelling: the state space

State space - `gym.spaces.Box(low=-2, high=8, shape=H×W)`: The state space of the custom MDP corresponds to what the agent actually *sees* when playing. A cell in the *player board* could be in the following states:

- -2: the cell is unrevealed
- -1: this is **not actually a possible state** for the agent. It's only the way in which a mine is represented in the board containing the *solution* of the game.
  The agent is informed about losing a game by the `terminated` boolean variable of the *Gym Env*.
  The agent is never informed about the location of the mines.
- 0...8: a revealed safe cell. The number represents the mines around the cell.

MDP modelling: the reward (first version)

It's inspired by the paper Wang and Lei (2025).

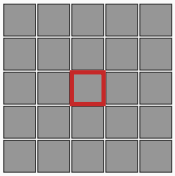
Outcome	**Condition**	**Reward**
Win	num. safe cells == num uncovered cells	$+H \cdot W$
Loss	mine selected	$-H \cdot W$
Progress	informed safe cell uncovered	$+1$
Guess	safe cell *discovered by chance*	-0.5
Already open	already-revealed cell selected	-0.5

However, during the learning phase, this design of reward presented **some issues**:

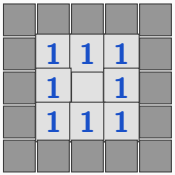
1. The agent learned to **remain** on an **already discovered** cell for many timesteps
2. During *a progression* more than one cell could be opened. However, the reward was still $+1$ **independently** on the number of the **opened cells**.
3. The overall reward was dependent on the board size.

Aside: Minesweeper's flood-fill

Before — click $(2, 2)$



After — cascade complete



- Imagine selecting a cell with no mines in its neighborhood (*i.e. a 0 cell*). According to the game rules, all the adjacent connected cells are revealed.
- Instead of relying on an inefficent recursive solution, we though about a iterative solution *stack-based*. The coords of the cells with no mine around are pushed as soon as they are found, and popped as soon as their boundaries are explored. No more pushes are done with one of this conditions: a mine is found, a numbered cell is found, the cell has already been uncovered by a previous visit.

MDP modelling: action masking + reward normalization

Action masking

To let the agent *risk* more, Q-values/logits are set to $-\infty$ before letting the agent perform the next action.

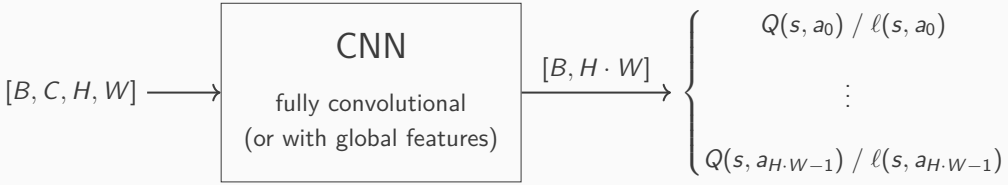
Reward normalization

With normalization, the rewards bounds are kept under control whatever is the size of the board game.

$$R_t = \begin{cases} -1 & \text{a mine is selected} \\ -\dfrac{0.5}{S} & \text{already unrevealed cell is selected} \\ \dfrac{\Delta n_t}{S} & \text{uncovering } \Delta n_t \text{ cells } \textbf{without guessing} \\ \dfrac{\Delta n_t}{S} - \dfrac{1.5}{S} & \text{uncovering } \Delta n_t \text{ cells } \textbf{with a guess action} \\ 1 + \dfrac{\Delta n_t}{S} & \text{winning } \textbf{without guessing} \\ 1 + \dfrac{\Delta n_t}{S} - \dfrac{1.5}{S} & \text{winning } \textbf{with a guess action} \end{cases}$$

Network architectures: state space encoding

- Our original idea on how to represent the state space is based on the paper by Wang and Lei (2025). The authors used a one-hot board encoding (10 channels: values 0–8 plus one channel for unrevealed cells) fed to a convolutional Q-network.



- In a second stage of development we added a constant channel, the 11^{th} one, containing the *mine density*. This was done with the idea to let the network generalize well to harder games. From now on, $C = 11$.

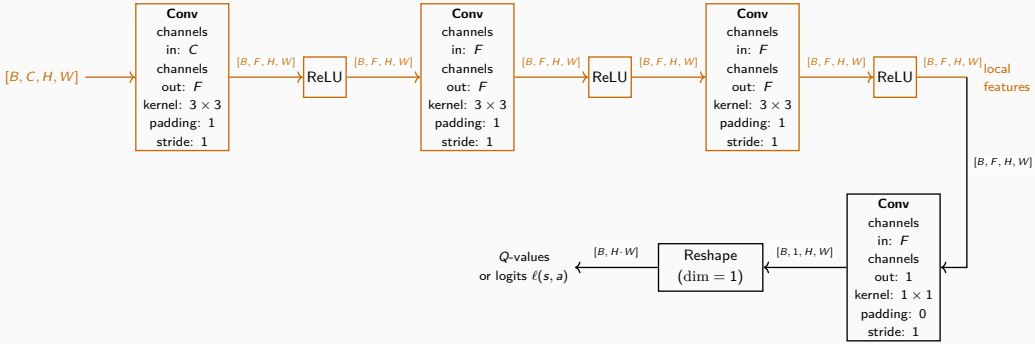
Design Choices for Stable Training

- `player_board` content is not re-usable as PyTorch index; otherwise, we might have the risk to override the wrong one-hot channel.

One-hot channel ID	Encoded value / meaning
$0, \ldots, 8$	Values $0, \ldots, 8$ of `player_board`
9	-2: unrevealed cell
10	Mine density

- **Safe first move**: if the first action hits a mine, the board is regenerated from the same RNG (not reseeded) until the first move is safe. This avoids penalizing the agent only due to *its bad luck*.

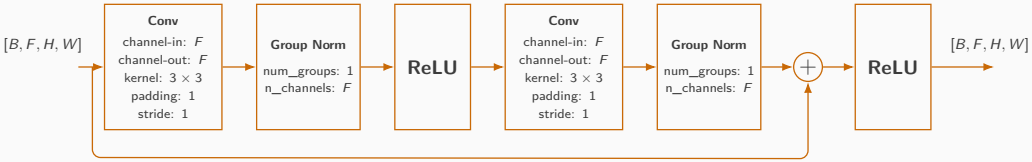
Architecture 1: Fully-convolutional CNN



Why fully convolutional?

A 1×1 conv head produces one scalar per cell ⇒ the same weights work for any $H \times W$ board. A feed-forward classification head would have the issue to create space for a W matrix whose shape depend on the board size.

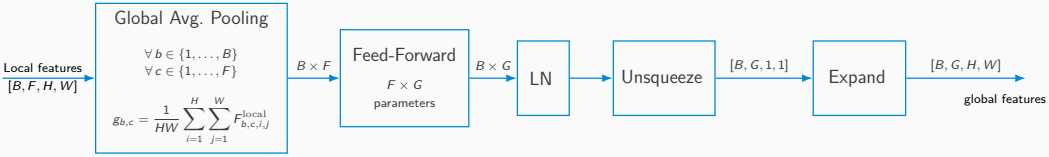
Architecture 2: *Global-skip* architecture - the residual block



Why a group norm?

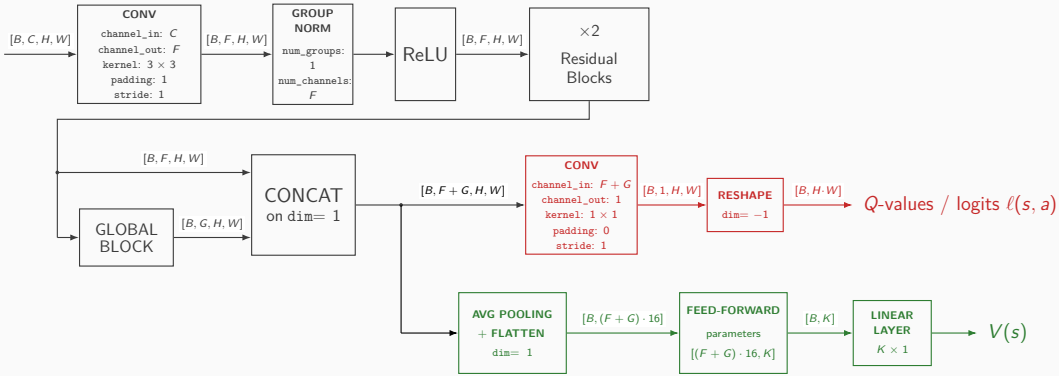
The GN is commonly used with 4D tensors in CV, and for our purposes it allowed us to maintain the independence on the size of the grid. With `num_groups` $= 1$, the BN is mathematically equivalent to a LN layer.

Architecture 2: *Global-skip* architecture - the global block



The section of the NN producing the local features should match local patterns around the cell. The global section now presented takes in input the output of the convolutional backbone and should generalize the status of the entire board. A global average pooling is used, preserving a $4x4$ space dimension. This is a compromise between loosing completely the spatial dimension and loosing completely the independence on the board shape.

Architecture 2: *Global skip* architecture - the overall view



DRL algorithms: DQN

- online Q_θ + target Q_{θ^-} network (two replicas of same architecture, weights θ become θ^- every `target_update_frequency`)
- replay buffer, capacity `replay_buffer_capacity`, uniform sampling of a batch made of `batch_size` transitions
- ε-greedy action selection, masked over already revealed cells. Could be linearly or exponentially decayed (`epsilon_decay_type`) until `final_epsilon`

Huber Loss

- It's less sensitive to huge TD errors during the first optimization steps.
- Gradient clipping to prevent exploding gradients in backprop.

$$y = R_{t+1} + \gamma(1 - \mathit{terminated}) \max_{a' \,:\, \text{masked}} Q_{\theta^-}(s', a')$$

$$\delta_{\mathrm{TD}} = y - Q_\theta(s, a)$$

$$\mathcal{L}(\theta) = \begin{cases} \frac{1}{2}\delta_{\mathrm{TD}}^2, & |\delta_{\mathrm{TD}}| \le 1, \\ |\delta_{\mathrm{TD}}| - \frac{1}{2}, & |\delta_{\mathrm{TD}}| > 1. \end{cases}$$

DRL algorithms: PPO - components

- rollout buffer: A rollout is a sequence of iteractions between an agent and the environment **collected under the same current policy**. It can be imagined as *watching the agent playing the current game* and recording what is does. Its usage is a common practice with PPO algorithms according to Stable-Baselines3.
- separate **actor** and **critic** networks, separate Adam optimizers.
- Shared CNN backbones as seen before, with the difference that the critic's value head outputs a single $V(s)$.
- independent gradient-clip norms.
- both the learning rate and gradient norms are parametrized as: `max_actor_grad_norm`, `max_critic_grad_norm`, `actor_learning_rate`, `critic_learning_rate`.

DRL algorithms: PPO - main loops (Schulman, Wolski, et al. 2017)

- **Rollout phase**: Inside a rollout buffer, transitions are stored for $T =$ `rollout_buffer` consectutive transitions, independently from the fact that these are transaction of the same or different episodes. When we encounter the end of an episode, we continue to insert transitions into the rollout unless: we arrive at T or we reach the total number of episodes / `truncated` Gym signal.
- **Learning phase**: The agent learns from the data collected during the rollout phase. For each epoch, the whole content of the rollout buffer is divided in mini-batch containing `batch_size` transitions. For each of these mini-batches, the actor and critic weights are updated. The whole process is repeated for $K =$ `update_epochs`, at the end of those the whole rollout buffer is emptyed.

Generalized Advantage Estimation (Schulman, Moritz, et al. 2016)

$c_t = 0$ when `terminated` or `truncated` $= 1$.

$$G_t := R_t + \gamma c_t G_{t+1}$$

$$A_t := G_t - V(S_t)$$

$$\delta_t^{TD} := R_{t+1} + \gamma c_t V(S_{t+1}) - V(S_t)$$

By adding the quantity $(\gamma c_t V(S_{t+1}) - \gamma c_t V(S_{t+1}))$ in the advantage equation:

$$A_t := R_t + \gamma c_t G_{t+1} - V(S_t) + \gamma c_t V(S_{t+1}) - \gamma c_t V(S_{t+1})$$

$$= R_t + \gamma c_t V(S_{t+1}) - V(S_t) + \gamma c_t G_{t+1} - \gamma c_t V(S_{t+1})$$

$$A_t = \delta_t^{TD} + \gamma c_t A_{t+1} \rightarrow A_t = \delta_t^{TD} + \gamma \lambda c_t A_{t+1}$$

By introducing a parameter $\lambda \in [0, 1)$ we can relax Monte-Carlo for a multiple step TD estimation of G_t. With $\lambda = 0$, the advantage corresponds to TD(0), with $\lambda = 1$ we have full Monte-Carlo. A_t **are normalized** to reduce the impact of their variance.

Clipped Surrogate Objective

$$r_t(\theta) = \exp\big(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\big)$$

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\left(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\Big]$$

$$\mathcal{L}^{\text{actor}} = -\mathcal{L}^{\text{CLIP}} - \beta\mathcal{H}[\pi_\theta] \qquad \mathcal{L}^{\text{critic}} = \mathit{Huber}\big((V_\phi(s_t), \text{value target}_t)\big)$$

$$\mathcal{H}(\pi_\theta) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$$

The entropy loss is added to the actor to avoid falling into a sub-optimal deterministic policy too early. Low values of entropy suggests deterministic policies, high values suggest more stochastic policies.

`clip_epsilon` ϵ: keeps policy updates small on a discrete action space of size $H \cdot W$.
`entropy_coefficient` β: regularization parameter tuning the convergence to a deterministic/stochastic policy.

Experimental Setup

- Initial experiments were conducted with board 6×6, 5 mines (density $\approx 13.9\%$) identical for DQN and PPO. The metric we aim to maximize is the `win_rate`, defined as number of episodes in which the agent won over the total number of test episodes. In this first phase, we were able to achieve a max win rate of $\approx 80.1\%$ for DQN and $\approx 80.2\%$ for PPO.
  Further experiments were conducted with boards 9×9 and 10 mines.
- disjoint RNG seed ranges for train / validation / held-out test
- final evaluation: 1000 held-out test episodes per agent
- to conduct in a rigorous and ordered manner an experimental performance analysis, we used the Weight and Biases platform (academic plan) and its Python Library.

Hyperparameters Tuning

Search pipeline (exploiting W&B Sweeps)

- `bayes` search: samples guided by past trials, faster convergence than random search
- Hyperband Early Termination: kills worst trials at each validation checkpoint, saving time for promising configs
- **Screening** $\rightarrow$ many trials, episode budget scaled to board size
- **Promotion** $\rightarrow$ best config retrained on 3 independent seeds (100k ep.) for robustness
- Due to time and computational constraints, our search focused on **9x9@10 mines** boards only.

Search space, by parameter nature

- *log-uniform*: `learning_rate` $\in [10^{-4}, 10^{-3}]$
- *continuous uniform*:
  `clip_epsilon` (PPO) $\in [0.1, 0.3]$,
  `epsilon_beta` (DQN) $\in [0.999, 0.9998]$
- *integer uniform*:
  `update_epochs` (PPO) $\in [3, 10]$
- *discrete grid*: `discount_factor`
  $\in \{0.8, 0.9, 0.95, 0.99\}$,
  `replay_buffer_size` (DQN)
  $\in \{50000, 100000\}$,
- *categorical*: `epsilon_decay_type` (DQN)
  $\in \{\text{linear}, \text{exponential}\}$

Statistical tests: Fixing DRL algorithm, `fully_conv` VS `global_skip_conv`

Due to the nature of the environment, the metric we aimed to maximize is the win rate. Statistical test used: McNemar test; it's the equivalent of the Wilcoxon-signed rank test for binary events. The test focuses only on disagreeing episodes.

$c =$ `num_episodes` won by B $\quad$ b `num_episodes` won by A.

`p_value` = Probability of observing c wins of B over $b + c$ total victories

$= P(X = c)$, $X \sim \text{Binom}(n = b + c, p = 0.5)$. $\mathcal{H}_0 : P(A : wins, B : lose) = P(A : lose, B : wins)$

DQN

`agent_seed`	Win rate (`fully_conv`)	Win rate (`global_skip`)	p-value	Significant ($\alpha = 0.05$)
43	**69%**	63%	3.32×10^{-3}	yes
44	67%	67%	0.88	no
45	68%	67%	0.60	no

PPO

`agent_seed`	Win rate (`fully_conv`)	Win rate (`global_skip`)	p-value	Significant ($\alpha = 0.05$)
43	62%	**72%**	7.48×10^{-8}	yes
44	65%	67%	0.28	no
45	56%	**65%**	1.68×10^{-6}	yes

Statistical tests: Fixing architecture, DQN vs PPO

`fully-conv`

`agent_seed`	Win rate (DQN)	Win rate (PPO)	p-value	Significant ($\alpha = 0.05$)
43	**69%**	62%	8.23×10^{-4}	yes
44	67%	65%	0.21	no
45	**68%**	56%	8.26×10^{-10}	yes

`global-skip`

`agent_seed`	Win rate (DQN)	Win rate (PPO)	p-value	Significant ($\alpha = 0.05$)
43	63%	**72%**	1.52×10^{-6}	yes
44	67%	67%	1	no
45	67%	65%	0.36	no

Observations:

- There's a strong statistical evidence about the sinergy between PPO and the `global_skip` architecture.
- DQN algorithm performs *slightly better* by using `fully_conv` architecture, even though there's not a strong statistical evidence.

Note: each test run has been performed on the same held-out test episodes: that's the reason why paired statistical tests have been used.

Future works

During this project we reasoned a lot how to generalize to wider boards. Basing on what we've studied until now, we put our maximum effort to train and use at inference time the networks independently on the board size.

Future work might include:

- *curriculum learning*: models learn gradually, mimicking how humans learn (starting from the basics and moving on to complex topics). The idea would be graudally adapt the learning starting from little to bigger and more *mine dense* boards.
- *self-attention* in global section of the `global_skip_connection` architecture. A self-attention layer would allow to each cell to measure the impact of all the other cells of the boards, not only to the ones of its neighboroud. It might preserve spatial features better than a 4x4 global average pooling.

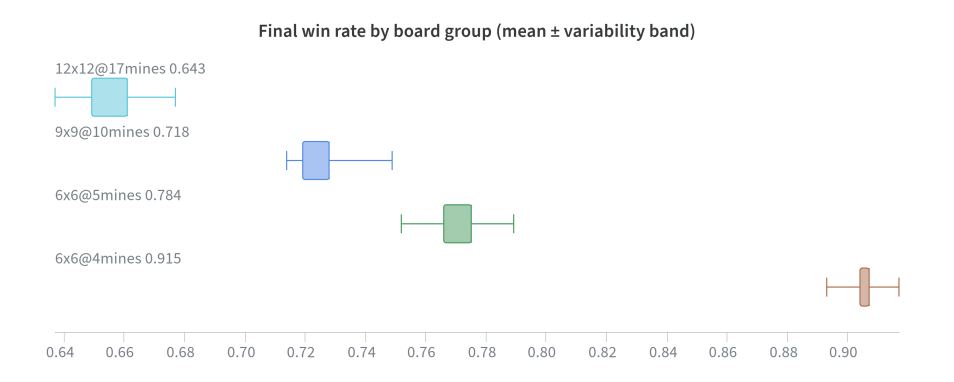
AI acknowledgments

- The source code interacting with W&B to store the complete run logs and manage the Sweeps was produced with the help of Claude Opus 4.6 and Gemini 3.5 Flash.
- General code cleaning and bug fixing was carried out with the help of Claude Sonnet 5.
- The LaTeX code in charge of producing TikZ drawings for neural architecture blocks or other drawings has been produced with the help of GPT 5.6-Sol and Gemini Flash 3.5.

References

- Schulman, J., P. Moritz, et al. (2016). **"High-Dimensional Continuous Control Using Generalized Advantage Estimation".** In: *arXiv preprint arXiv:1506.02438*.
- Schulman, J., F. Wolski, et al. (2017). **"Proximal Policy Optimization Algorithms".** In: *arXiv preprint arXiv:1707.06347*.
- Wang, W. and C. Lei (2025). **"Training a Minesweeper Agent Using a Convolutional Neural Network".** In: *Applied Sciences* 15.5, p. 2490. ISSN: 2076-3417. DOI: 10.3390/app15052490. URL: https://www.mdpi.com/2076-3417/15/5/2490.

Backup Slides: Confidence Intervals for Best Model (PPO Global-Skip Conv)



board group	mean win rate	95% CI
6x6@4mines	0.9068	[0.9023, 0.9113]
6x6@5mines	0.7731	[0.7655, 0.7807]
9x9@10mines	0.7291	[0.7219, 0.7363]
12x12@17mines	0.6569	[0.6494, 0.6644]

Backup Slides: Train and Inference Time

Algorithm	Architecture	Training Time	Inference Time
DQN	FULLY-CONV	$\sim$ 1 h	6 s
PPO	FULLY-CONV	$\sim$ 25 min	7 s
DQN	GLOBAL-SKIP	$\sim$ 1 h 45 min	10 s
PPO	GLOBAL-SKIP	$\sim$ 30 min	11 s